



Pool de Threads & Ejecutores

Pool de Threads



Un *pool de threads* es un **patrón de diseño** para gestionar hilos de forma eficiente evitando crear y destruir hilos constantemente, ya que esas operaciones son caras a nivel del sistema operativo.

Explicación

Cuando un programa crea un hilo en Java:

- La JVM pide al **sistema operativo** que cree un hilo nativo.
- Esto es **costoso** (memoria, cambio de contexto, inicialización...).
- Destruir un hilo también consume recursos.

Si lanzamos cientos o miles de hilos con:

```
for (...) {  
    new Thread(tarea).start();  
}
```

...la aplicación puede saturar el SO muy rápido.

El patrón **Thread Pool** resuelve este problema manteniendo un **conjunto de hilos ya creados**, que se reutilizan para ejecutar muchas tareas distintas.

¿Qué aporta?

- Reduce la sobrecarga de creación y destrucción.
- Controla y limita el número de hilos.
- Evita que tu aplicación cree hilos sin límite.

- Separa la **definición de la tarea** de la **gestión de los hilos** → más limpio y escalable.

¿Cómo funciona internamente?

Un *thread pool* contiene:

1. **Una cola de tareas** pendientes (work queue).
2. **Un conjunto de hilos trabajadores** (workers).

El flujo es:

- Tú envías una tarea al pool.
- La tarea entra en la cola.
- Alguno de los hilos disponibles la ejecuta.
- Cuando termina, el hilo **no muere**: vuelve a quedar disponible para otra tarea.

Concepto clave

Un thread pool no ejecuta código, ejecuta tareas (Runnable/Callable).

Los hilos son gestionados y reutilizados automáticamente.

Por eso, en un pool **nunca usamos** `.start()` .

Los hilos ya están arrancados o los arranca el executor internamente.

Ejecutores



Los *Ejecutores* son la **implementación en Java** del patrón Thread Pool.

- Abstracción → ejecutor.
- Implementación → thread pool

Explicación

A través de este framework podemos desacoplar completamente la ejecución de una tarea de la forma en la que se gestiona el ciclo de vida de ese hilo.

Tú entregas "tareas" (Runnable/Callable).

El Executor decide **cuándo** y **en qué hilo** se ejecutan.

Nosotros solo tenemos que encargarnos de montar el ejecutor que sea, y de implementar la unidad de tarea, y será el ejecutor quien internamente cambie el estado del hilo.

El framework de concurrencia de Java define varias interfaces:

- **Executor**
- **ExecutorService**
- **ScheduledExecutorService**
- **ThreadPoolExecutor** (implementación configurable)
- Métodos fábrica de `Executors` (`newFixedThreadPool` , etc.)

Su interfaz más básica tiene esta pinta:

```
public interface Executor {  
    void execute(Runnable command);  
}
```

```
Executor e = command → new Thread(command).start();  
e.execute() → System.out.println("Hola");
```

Executor

Es la interfaz más simple:

```
public interface Executor {  
    void execute(Runnable command);  
}
```

Ejemplo conceptual:

```
Executor e = r → new Thread(r).start();
```

```
e.execute(() → System.out.println("Hola"));
```

Executor **solo sabe ejecutar una tarea**, pero no da herramientas de gestión del ciclo de vida.

ExecutorService

Esta es la que **sí** usarás para pools reales.

Permite:

- enviar tareas (`execute` , `submit`). Enviamos y ya el ejecutor las ejecutará cuando quiera.
- recibir resultados (`Future`)
- apagar el pool (`shutdown` , `shutdownNow`)
- esperar a que termine (`awaitTermination`)

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
executor.execute(() → System.out.println("Hello"));  
executor.shutdown();
```

Importante

- `shutdown()` → no acepta más tareas; NO BLOQUEA.
- `awaitTermination()` → bloquea hasta que termine el pool.

Siempre que uses un pool debes **cerrarlo**.

Tipos de Pool más comunes

Java ofrece varios "preconfigurados", aunque todos son en realidad distintas configuraciones de `ThreadPoolExecutor`.

`newFixedThreadPool(n)`

Pool con **n hilos fijos**.

- Si hay más tareas que hilos → se encolan.

- No se crean hilos nuevos.
- Se reutilizan siempre los mismos.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
```

Ideal para:

- servidores web,
- procesamiento estable,
- limitar la carga del sistema.

newCachedThreadPool()

Pool **dinámico**:

- Crea hilos nuevos si hacen falta.
- Reutiliza hilos inactivos.
- Si no hay tareas → los hilos expiran.

```
ExecutorService pool = Executors.newCachedThreadPool();
```

Ideal para:

- muchas tareas cortas,
- picos impredecibles de carga.

No recomendado si no quieres crecimiento descontrolado del número de hilos.

newSingleThreadExecutor()

Usa **solo un hilo**.

- Garantiza orden de llegada (FIFO).
- Garantiza exclusión mutua (solo 1 hilo ejecuta tareas).

```
ExecutorService single = Executors.newSingleThreadExecutor();
```

Ideal para:

- tareas secuenciales,
- logs,
- actualizaciones que deban hacerse en orden.

ThreadPoolExecutor (configurable)

El más completo. Permite definir:

- corePoolSize
- maximumPoolSize
- keepAliveTime
- workQueue
- política de rechazo, etc.

Dependiendo de los parámetros que le demos podemos tener cualquiera de los otros tipos, ya que los otros tipos no son más que ExecutorsServices preconfigurados.

Si por ejemplo le damos `corePoolSize = maxPoolSize`, tendremos un fixed.

Ejemplo de fixed personalizado:

```
ExecutorService pool = new ThreadPoolExecutor(  
    4,          // core  
    4,          // máximo  
    0L, TimeUnit.MILLISECONDS,  
    new LinkedBlockingQueue<>()  
);
```